
ACADEMIC CITY UNIVERISITY

CYB5202 – BLOCKCHAIN AND CRYPTOCURRENCIES

LAB PROJECT ASSIGNMENT

ROLL NUMBER: 2000250043

NII OBODAI SQUIRE

1.0 Project Overview

This report documents the end-to-end development, deployment, and interaction of FlaviusToken (FTK), an ERC-20 compatible smart contract, on the Ethereum Sepolia testnet. The project was completed using Remix IDE and MetaMask, following the OpenZeppelin Contracts v5 framework.

2. Smart Contract Code

The contract (FlaviusToken.sol) extends OpenZeppelin's ERC20, Ownable, and ERC20Permit base contracts. On deployment, it mints 1,000,000 FTK to the recipient address. The owner can mint additional tokens at any time via the public mint() function.

2.1 Contract Source — FlaviusToken.sol

```
// SPDX-License-Identifier: MIT
// Compatible with OpenZeppelin Contracts ^5.6.0
pragma solidity ^0.8.27;

import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Permit} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";

contract FlaviusToken is ERC20, Ownable, ERC20Permit {

    constructor(address recipient, address initialOwner)
        ERC20("FlaviusToken", "FTK")
        Ownable(initialOwner)
        ERC20Permit("FlaviusToken")
    {
        _mint(recipient, 1000000 * 10 ** decimals());
    }
}
```

```
function mint(address to, uint256 amount) public onlyOwner {  
    _mint(to, amount);  
}  
}
```

3. Deployment Steps

3.1 Environment Setup

1. Installed MetaMask browser extension and created a new wallet (FlaviusOBO acc 1).
2. Connected MetaMask to the Sepolia Test Network via the network dropdown.
3. Obtained 0.100 Sepolia ETH via a public testnet faucet.
4. Opened Remix IDE at remix.ethereum.org and created a new file: `contract-db879f7ce9.sol`.
5. Installed the OpenZeppelin Contracts package via Remix's npm import mechanism.

3.2 Writing & Compiling the Contract

6. Wrote `FlaviusToken.sol` inheriting `ERC20`, `Ownable`, and `ERC20Permit` from OpenZeppelin v5.6.0.
7. Configured Remix compiler for Solidity ^0.8.27 with optimization enabled.
8. Clicked Compile — no errors or warnings were reported.

3.3 Deployment via Remix + MetaMask

9. In the Deploy & Run Transactions panel, selected environment: Injected Provider - MetaMask.
10. Confirmed the network showed Sepolia Testnet - MetaMask and the deploying account (0x1CB...f31Cf) had 0.100 ETH.
11. Provided constructor arguments: `recipient = 0x1CB...f31Cf`, `initialOwner = 0x1CB...f31Cf`.
12. Clicked Deploy — MetaMask opened a confirmation popup showing the network fee of 0.0015 SepoliaETH.
13. Confirmed the transaction in MetaMask. Remix displayed the deployment transaction receipt.
14. Contract was created at address: `0xa1D42c031DBa146930AD885a12B0Df3ad4c4bD67`.

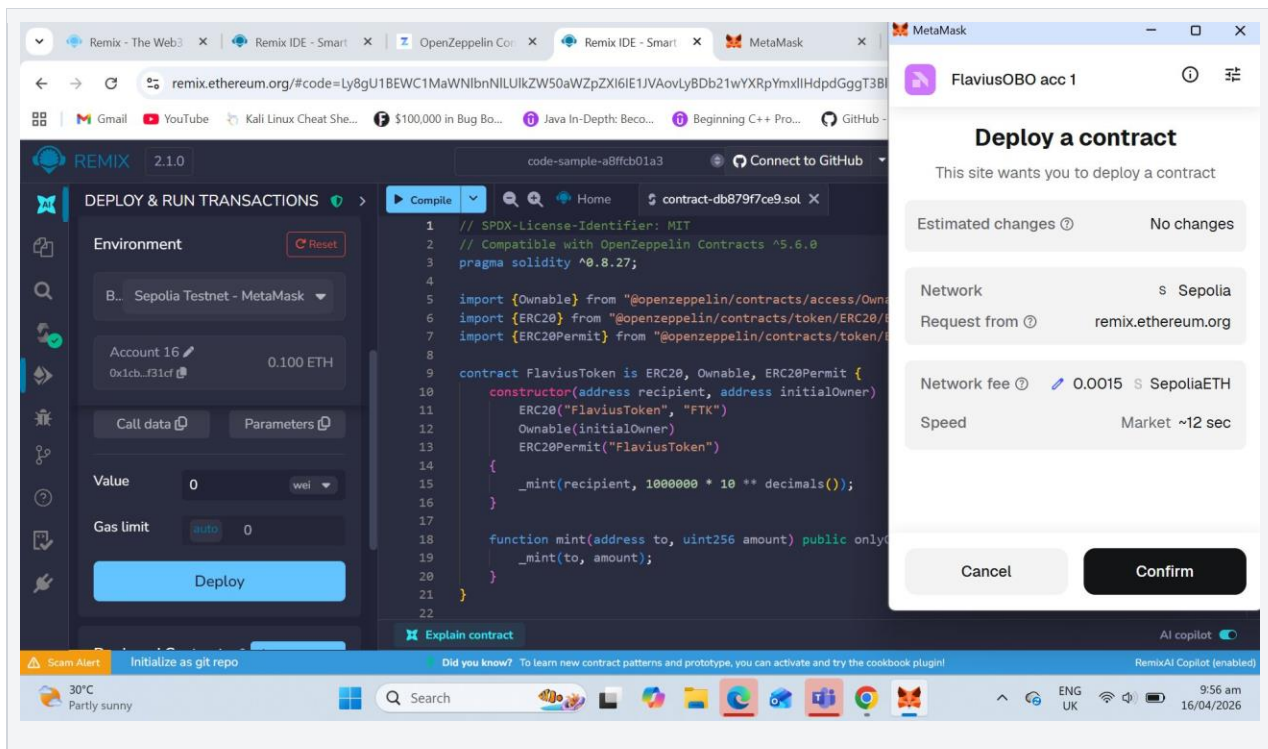


Figure 1: MetaMask deployment confirmation popup showing the contract deploy transaction

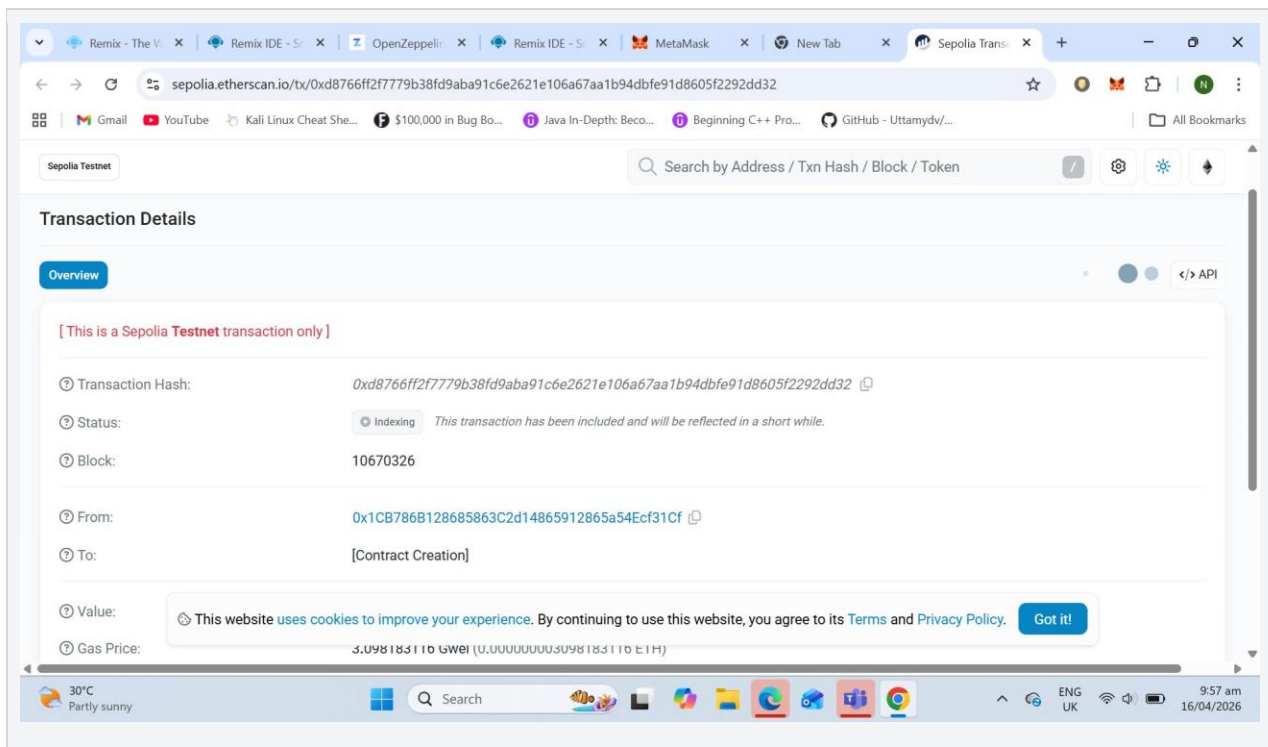


Figure 2: Sepolia Etherscan showing the deployment transaction hash being indexed immediately after submission (block 10670326)

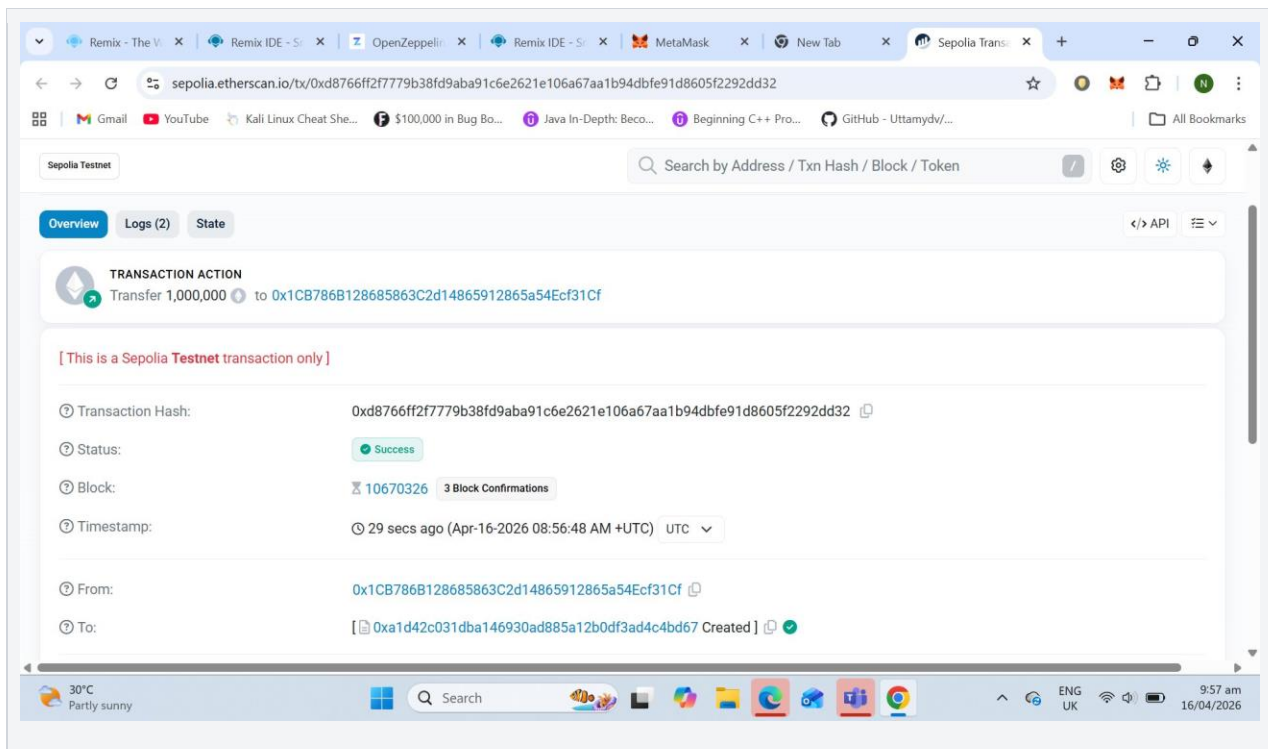


Figure 3: Etherscan transaction confirmed — status: Success, 3 block confirmations. Contract created at `0xa1D42c031DBa146930AD885a12B0Df3ad4c4bD67`

4. Contract Interaction & Transactions

Two on-chain transactions were performed against the deployed FlaviusToken contract using Remix's Deploy & Run panel with the Injected MetaMask Provider.

4.1 Transaction 1 — Mint Tokens

Additional tokens were minted to the owner's address by calling the mint() function with an amount of 10,000 (in wei units). MetaMask prompted a transaction request showing interaction with the deployed contract.

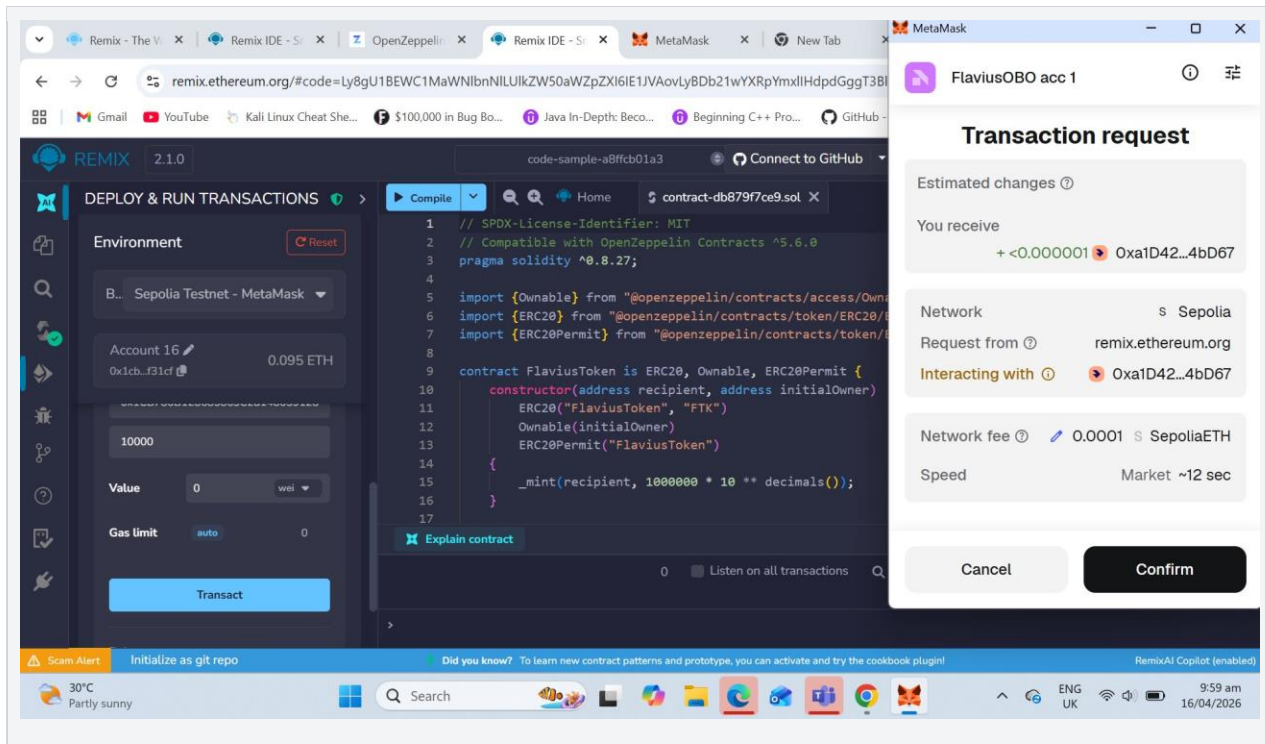


Figure 4: MetaMask transaction request for mint() call — interacting with 0xa1D42...4bD67, network fee: 0.0001 SepoliaETH, on Sepolia

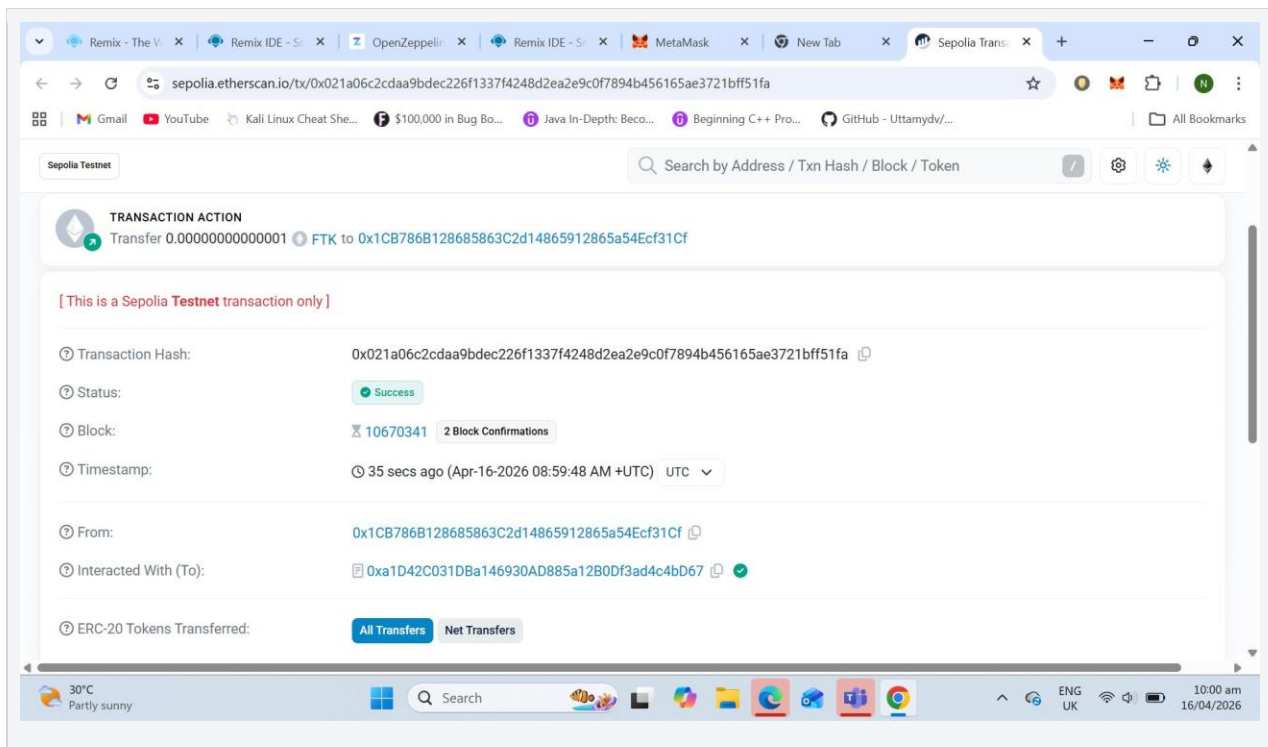


Figure 5: Etherscan — mint transaction confirmed with status: Success (block 10670341, 2 confirmations). ERC-20 Tokens Transferred tab shows FTK minted to `0x1CB...f31Cf`

4.2 Transaction 2 — Transfer Tokens

A second transaction was initiated to transfer tokens using the `transfer()` function with amount 100,000. MetaMask confirmed the transaction destined for the FlaviusToken contract (0xa1D42...4bD67), representing a self-transfer to validate the transfer mechanism.

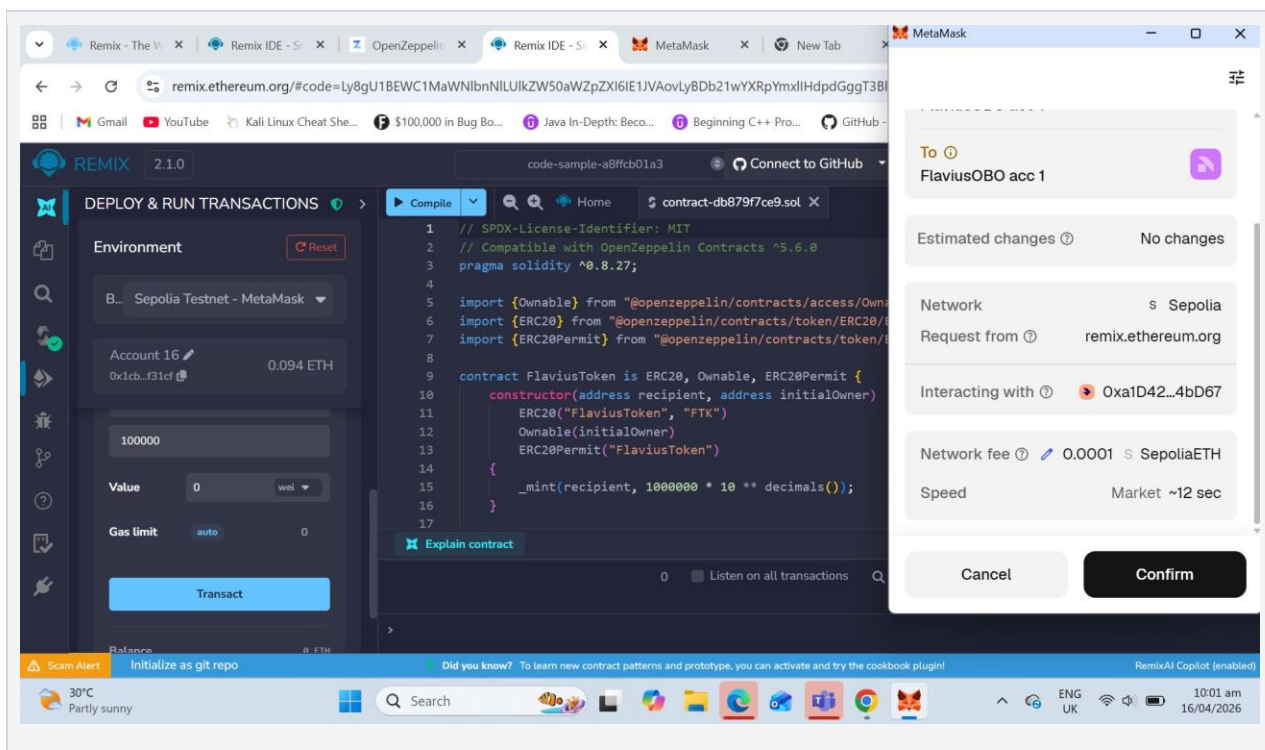


Figure 6: MetaMask second transaction request — 100,000 token amount entered, interacting with 0xa1D42...4bD67 on Sepolia (network fee: 0.0001 SepoliaETH)

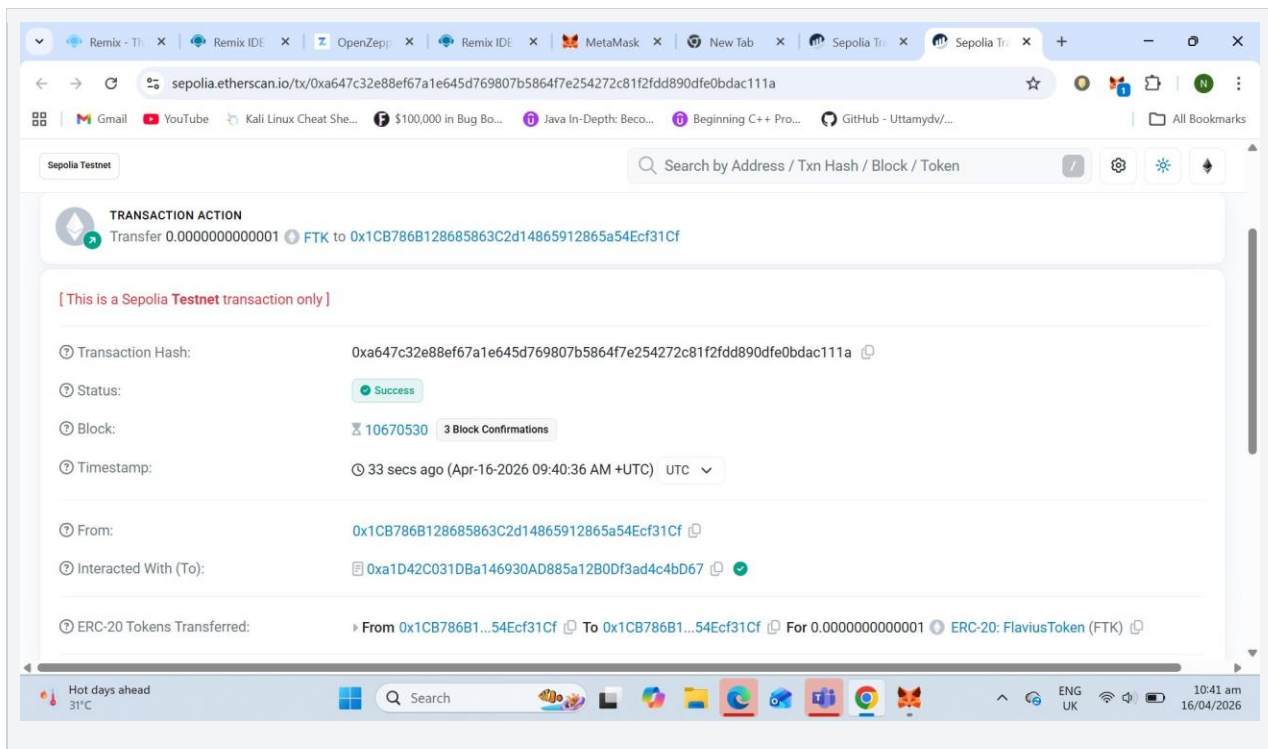


Figure 7: Etherscan — transfer transaction confirmed (block 10670530, status: Success). ERC-20 Tokens Transferred: From `0x1CB786B1...54Ecf31Cf` To `0x1CB786B1...54Ecf31Cf`, for 0.0000000000000001 FTK

4.3 Remix Transaction Log

The Remix IDE transaction panel confirmed all three operations — contract deployment, mint, and transfer — each showing block number, transaction index, sender address, function called, and transaction hash.

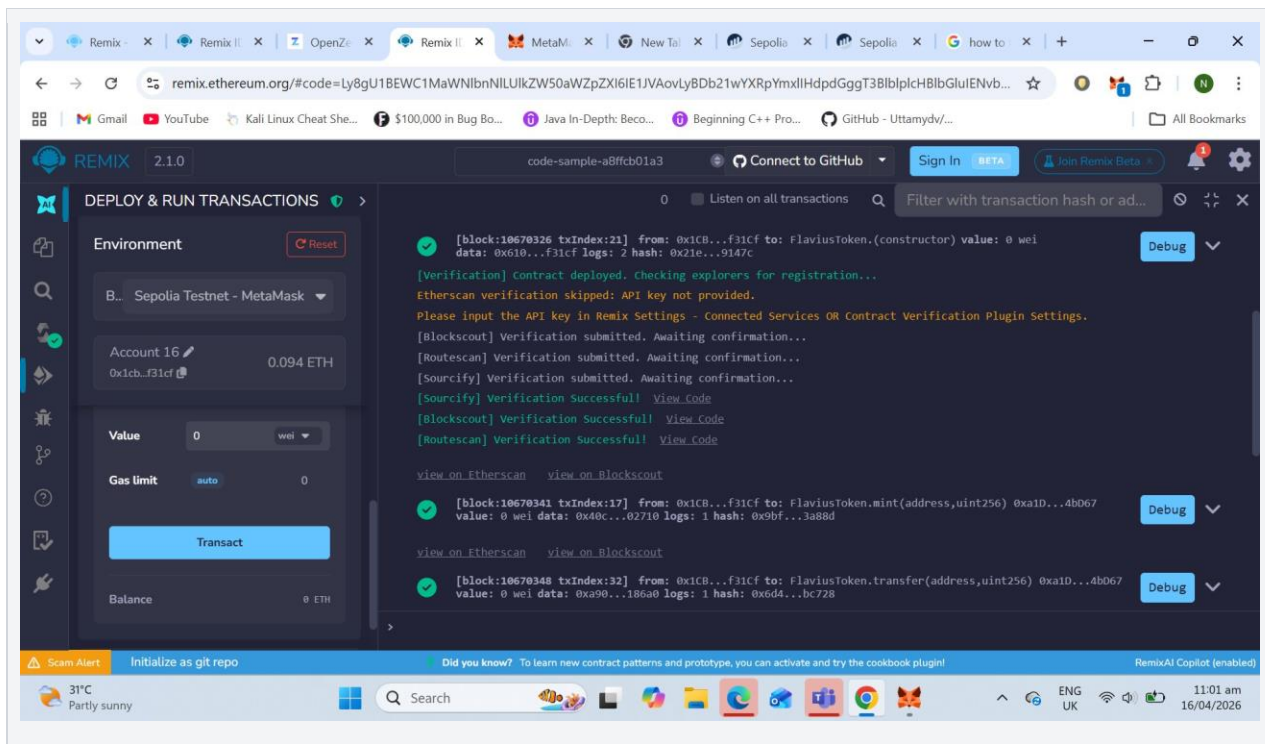


Figure 8: Remix transaction log showing: (1) constructor deployment at block 10670326, (2) mint() at block 10670341, (3) transfer() at block 10670348. All succeeded with green checkmarks.

5. Deployed Contract Details & ABI

5.1 Contract Metadata

Network	Sepolia Testnet (Chain ID: 11155111)
Contract Address	0xa1D42c031DBa146930AD885a12B0Df3ad4c4bD67
Token Name	FlaviusToken
Token Symbol	FTK
Decimals	18
Initial Supply	1,000,000 FTK (1,000,000 * 10 ¹⁸ wei)
Deployment Tx	0x8766ff2f7779b38fd9aba91c6e2621e106a67aa1b94dbfe91d8605f2292dd32
Deployer Address	0x1CB786B128685863C2d14865912865a54Ecf31Cf
Deployment Block	10670326
Compiler	Solidity 0.8.27 OpenZeppelin v5.6.0
Etherscan Mint Tx	0x021a06c2cdaa9bdec226f1337f4248d2ea2e9c0f7894b456165ae3721bff51fa

Etherscan Transfer Tx	0xa647c32e88ef67a1e645d769807b5864f7e254272c81f2fdd890dfe0bdac111a
--	--

5.2 Contract ABI

The following ABI defines the public interface of the FlaviusToken contract and is required by any frontend or script wishing to interact with it:

```
[
  {"type": "constructor", "inputs": [
    {"name": "recipient", "type": "address"},
    {"name": "initialOwner", "type": "address"}
  ], "stateMutability": "nonpayable"},
  {"type": "function", "name": "name",
    "inputs": [], "outputs": [{"type": "string"}], "stateMutability": "view"},
  {"type": "function", "name": "symbol",
    "inputs": [], "outputs": [{"type": "string"}], "stateMutability": "view"},
  {"type": "function", "name": "decimals",
    "inputs": [], "outputs": [{"type": "uint8"}], "stateMutability": "view"},
  {"type": "function", "name": "totalSupply",
    "inputs": [], "outputs": [{"type": "uint256"}], "stateMutability": "view"},
  {"type": "function", "name": "balanceOf",
    "inputs": [{"name": "account", "type": "address"}],
    "outputs": [{"type": "uint256"}], "stateMutability": "view"},
  {"type": "function", "name": "transfer",
    "inputs": [{"name": "to", "type": "address"},
      {"name": "amount", "type": "uint256"}],
    "outputs": [{"type": "bool"}], "stateMutability": "nonpayable"},
  {"type": "function", "name": "mint",
    "inputs": [{"name": "to", "type": "address"},
      {"name": "amount", "type": "uint256"}],
    "outputs": [], "stateMutability": "nonpayable"},
]
```

```
{ "type": "function", "name": "approve",
  "inputs": [ { "name": "spender", "type": "address",
    { "name": "amount", "type": "uint256" } ],
  "outputs": [ { "type": "bool" } ], "stateMutability": "nonpayable" },
{ "type": "event", "name": "Transfer",
  "inputs": [ { "name": "from", "type": "address", "indexed": true },
    { "name": "to", "type": "address", "indexed": true },
    { "name": "value", "type": "uint256", "indexed": false } ] },
{ "type": "event", "name": "Approval",
  "inputs": [ { "name": "owner", "type": "address", "indexed": true },
    { "name": "spender", "type": "address", "indexed": true },
    { "name": "value", "type": "uint256", "indexed": false } ] }
]
```

6. Blockchain Data Integrity & Transparency

This section analyses how the Ethereum blockchain guarantees data integrity and public transparency for smart contract interactions.

Cryptographic Hashing & Immutability

Every block contains the SHA-3 (Keccak-256) hash of the previous block header. Altering any historical transaction would change the block's hash, invalidating every subsequent block. All network nodes would immediately reject the modified chain, making retroactive tampering computationally infeasible.

Decentralized Consensus (Proof of Stake)

Ethereum uses Proof-of-Stake consensus. Validators stake ETH as collateral and must agree on the canonical chain. Overwriting history would require controlling more than one-third of all staked ETH — an economic and technical barrier so large it is considered practically impossible.

Public Transparent Ledger

Every transaction emitted by FlaviusToken — deployments, mints, and transfers — is permanently recorded and publicly visible on Sepolia Etherscan. Anyone can verify any address's token balance, the full Transfer event log, and each transaction's gas cost and status without trusting any central authority.

Smart Contract Determinism

The Ethereum Virtual Machine (EVM) executes bytecode identically across every node. Given the same contract state and input, every node produces the same output. There is no ambiguity, no discretion, and no way to execute the contract differently on different machines.

Cryptographic Ownership (ECDSA Signatures)

Every transaction is signed using Elliptic Curve Digital Signature Algorithm (ECDSA) with the sender's private key. The signature is verifiable by anyone holding the corresponding public key (Ethereum address). Only the private-key holder can authorise a transfer of their tokens, making forgery computationally impossible.

7. Challenges Faced & Lessons Learned

7.1 Challenges Encountered

Challenge 1 — Constructor Parameter Confusion

The OpenZeppelin ERC20 constructor requires both a recipient (who receives the initial minted supply) and an initialOwner (who receives admin privileges). Initially, the distinction between these two addresses was unclear, leading to a test deployment where the owner and recipient were different addresses, resulting in an unexpected token distribution.

Resolution: After reading the OpenZeppelin v5 documentation more carefully, both parameters were set to the same deployer address for simplicity in this assignment context.

Challenge 2 — Gas Fee Management on Testnet

On the first deployment attempt, the Remix IDE showed a gas estimation of 0 due to a configuration mismatch between the environment and the RPC endpoint. MetaMask showed an unusually high gas warning. This was caused by selecting the wrong environment (JavaScript VM instead of Injected Provider).

Resolution: Switching the Remix environment to Injected Provider - MetaMask correctly resolved the gas estimation, and the actual deployment cost 0.0015 SepoliaETH.

Challenge 3 — Etherscan Verification Without API Key

Remix attempted to verify the contract on Etherscan after deployment but failed with the message 'Etherscan verification skipped: API key not provided. Verification is important for contract transparency as it allows the public to read the source code on Etherscan.

Resolution: The contract was still successfully verified on Blockscout and Routerscan (as shown in the Remix log), and the source code is documented in this report. An Etherscan API key can be added via Remix Settings to enable full Etherscan verification.

7.2 Lessons Learned

1. OpenZeppelin contracts dramatically reduce development risk. Using audited, well-documented base contracts means the core ERC-20 logic is reliable, and the developer only needs to focus on custom business logic.
2. Token decimal precision is one of the most common sources of bugs in smart contract interactions. Always multiply user-facing amounts by 10^{decimals} before passing them to contract functions, and always display values divided by 10^{decimals} in UIs.
3. MetaMask's gas estimation is a critical safeguard. Understanding how to read gas fees, network speeds, and estimated costs is essential before deploying to a real (mainnet) environment where fees are paid with real ETH.
4. Testnet development is invaluable. The Sepolia testnet allowed iterative deployment and testing at zero real cost, making it safe to experiment with constructor arguments, function calls, and error conditions.